

Real-Time Systems / Real-Time Operating Systems

ECE445M/ECE380L.12, Spring 2024

Midterm Exam

Date: March 7, 2024

UT EID: _____

Printed Name: _____
Last, First

Your signature is your promise that you have not cheated and will not cheat on this exam, nor will you help others to cheat on this exam:

Signature: _____

Instructions:

- Open book, open notes and open web.
- No electronic devices other than your laptop/PC (cell phones off and stowed away).
- You are allowed to access any resource on the internet, but no electronic communication other than with instructors.
- Please be sure that your answers to all questions (and all supporting work that is required) are contained in the space (boxes) provided.
- For all questions, unless otherwise stated, find the most efficient (time, resources) solution.

Problem 1	20	
Problem 2	20+5	
Problem 3	20	
Problem 4	20+5	
Problem 5	20	
Total	100+10	

Name: _____

Problem 1 (20 points): Miscellaneous

- a) Rank the debugging techniques based on the intrusiveness effect on the program, and explain your reasoning: Breakpoints, Logic Analyzer Pin Toggling, Array Dumps

- b) How can you detect and prevent stack overflow in your OS?

- c) Why can't you use spinlock semaphores with a priority scheduler?

- d) Why does PendSV need to be the lowest priority (priority = 7)?

Name: _____

Problem 2 (20+5 points): Blocking Semaphores

Assume a student in the class was trying to implement an OS using a round-robin scheduler with blocking semaphores using the following code, where the run list is implemented as a double linked list (with next and prev pointers in the TCB):

```
void OS_Wait(Sema4Type *semaPt) {
    semaPt->Value--;
    if (semaPt->Value < 0) {
        // Remove from run list
        RunPt->prev->next = RunPt->next;
        RunPt->next->prev = RunPt->prev;

        // Move to blocked list
        RunPt->next = semaPt->BlockPt;
        semaPt->BlockPt = RunPt;

        // Context switch
        OS_Suspend();
    }
}

void OS_Suspend() {
    ContextSwitch();
}
```

```
ContextSwitch
    BL PendSV_Handler
    BX LR

PendSV_Handler
    PUSH {R4-R11}
    LDR R0, =RunPt
    LDR R1, [R0]
    STR SP, [R1]
    LDR R1, [R1, #4]
    STR R1, [R0]
    LDR SP, [R1]
    POP {R4-R11}
    BX LR
```

Name: _____

Identify all the bugs in the code, explain what erroneous behavior will occur with the current implementation and how it could be fixed. Bonus points: extra credit for providing working code and fixes for the bugs by indicating changes to the code above.

Name: _____

Problem 3 (20 points): Synchronization 1

Observe the code below, running with standard blocking semaphores and a strict priority scheduler (lower number is higher priority):

<pre>void task1(){ OS_Wait(&s1); OS_Wait(&s2); printf("t1\n\r"); OS_Signal(&s2); OS_Kill(); }</pre>	<pre>void task2(){ OS_Wait(&s1); printf("t2\n\r"); ... OS_Kill(); }</pre>	<pre>void task3(){ OS_Wait(&s2); OS_Signal(&s1); OS_Signal(&s1); printf("t3\n\r"); OS_Signal(&s2); OS_Kill(); }</pre>
<pre>Sema4Type s1; Sema4Type s2; void main() { OS_Init(); OS_InitSemaphore(&s1, 0); OS_InitSemaphore(&s2, 1); OS_AddThread(&task1, 128, 1); OS_AddThread(&task2, 128, 2); OS_AddThread(&task3, 128, 3); OS_Launch(); }</pre>		

- a) Indicate the order in which tasks are interleaved in the code above by labeling each line of code in each task with ①, ②, ③, etc. in order of their execution, and show what the terminal output would look like.

- b) Assume *task2* runs for an excessive amount of time, is this behavior a problem for a real-time system? If so, what problems would it cause and what changes you could make to the OS to remedy this problem.

Name: _____

Problem 4 (20+5 points): Synchronization 2

Assuming an OS that implements spinlock semaphores and a strict priority scheduler (lower number is higher priority), observe the application code below:

```
void Task0(){ long sr;

    while(1) {

        sr = StartCritical();

        OS_bWait(&s1);

        // Toggle external LED
        // when data is updated

        GPIO_PORTF_DATA_R ^= 0x08;

        EndCritical(sr);

    }

}
```

```
void Task1(){

    while(1) {

        OS_bWait(&s1);

        // Print portf value
        // when data is updated

        printf("PF: %d\n\r", data);

        data = 0xDEADBEEF;

        OS_bSignal(&s1);

    }

}
```

```
void Task2(){

    OS_bWait(&s1);

    // sample data and notify

    data = GPIO_PORTF_DATA_R;

    OS_bSignal(&s1);

    OS_Kill();

}
```

```
Sema4Type s1;
uint32_t data;
void main() {

    OS_Init(); // will initialize ports
    data = 0xDEADBEEF;
    OS_InitSemaphore(&s1, 0);
    OS_AddThread(&Task0, 128, 1);
    OS_AddThread(&Task1, 128, 2);
    OS_AddPeriodicThread(&Task2, 1000, 2);
    OS_Launch();

}
```

There is at least one bug in the software. Identify all of bugs in the program and describe a potential solution that fixes them (including any followup issues). There may be less bugs than rows. Bonus points: extra credit for providing working code and fixes for the bugs as changes to the code above.

[illegible]

Name: _____

Problem 5 (20 points): Scheduling

Given a 3-task system with the following characteristics (periods and minimum/maximum execution times):

Task	Execution Time	Period
T1	5ms $\pm$ 5ms	40ms
T2	15ms $\pm$ 5ms	60ms
T3	10ms $\pm$ 10ms	120ms

- a) Is this taskset schedulable to meet all deadlines (equal to periods) using a fixed-priority scheduler with a rate-monotonic scheduling (RMS) scheme?

- b) What is the maximum, i.e. worst-case jitter (based on the start times of tasks) that can be experienced by each of the 3 tasks using an RMS strategy?

Name: _____

- c) Can you come up with a non-preemptive fixed/static schedule, i.e. a schedule in which tasks start at fixed, predetermined times and can not be interrupted in the middle of their execution to meet all deadlines while minimizing worst-case jitter among all tasks? Draw your schedule for one hyper-period. What is the worst-case jitter for each task?

